

# Data Analysis with Python

Advanced EDA & Introductory  
Statistical Analysis



# Topics:

- Data cleaning & transformation
- Relationship analysis
- Intro to probability & testing

“Today is about *insight*, not just visualization.”

# Why Clean Data Matters

- EDA (Exploratory Data Analysis) is only as good as the quality of your data.
- Real-world data often contains:
  - Missing values (NaN)
  - Duplicates
  - Outliers (extreme or unrealistic values)
  - Inconsistent formatting (e.g., “Male” vs “male”)
- If not handled, these issues lead to misleading results and false insights.

# Why Clean Data Matters

Showing a simple table comparing *raw* vs *cleaned* data.

| Name | Age | Salary |
|------|-----|--------|
| John | 29  | 50000  |
| Anna | NaN | 70000  |
| John | 29  | 50000  |
| Mike | 999 | 62000  |

After cleaning

| Name | Age | Salary |
|------|-----|--------|
| John | 29  | 50000  |
| Anna | 31  | 70000  |
| Mike | 32  | 62000  |

Show a simple table comparing *raw* vs *cleaned* data.

# Detecting Missing & Extreme Values

## Code Snippet:

```
# Check for missing values in each column  
df.isnull().sum()
```

## Output Example:

```
age      177  
fare      0  
cabin    687  
embarked  2
```

- `isnull().sum()` quickly reveals which variables need attention.
- Missingness patterns can reveal data collection issues.
- Columns with too many missing values may be dropped or imputed.

....

## **Visual Idea:**

- Bar chart showing number of missing values per column:

```
df.isnull().sum().plot(kind='bar', color='tomato')  
plt.title("Missing Values per Feature")  
plt.ylabel("Count")  
plt.show()
```

- “EDA is meaningless if your data is dirty — clean first, then explore.”

# Handling Missing Values

## **Techniques:**

- Drop missing: `df.dropna()`
- Fill with mean/median/mode

## **Code:**

```
df['age'].fillna(df['age'].median(), inplace=True)
```

- “Use domain logic when filling missing data.”

# Detecting Outliers (IQR Method)

## Concept:

- **Outliers** are values that are unusually high or low compared to the rest of the dataset.
- They can arise due to:
  - Data entry errors (e.g., extra zero)
  - Genuine rare events (e.g., very high income)
  - Measurement issues or anomalies

## Key Definitions:

- **Q1 (25th percentile):** Value below which 25% of data lies
- **Q3 (75th percentile):** Value below which 75% of data lies
- **IQR (Interquartile Range):**  $IQR = Q3 - Q1$
- **Outlier Rule:**
  - Values  $< Q1 - 1.5 \times IQR$  or  $> Q3 + 1.5 \times IQR$  are considered outliers



# Understanding Outliers and the IQR Method

Lower Bound =  $Q1 - 1.5 \times IQR$

Upper Bound =  $Q3 + 1.5 \times IQR$

- Example (explain step by step):

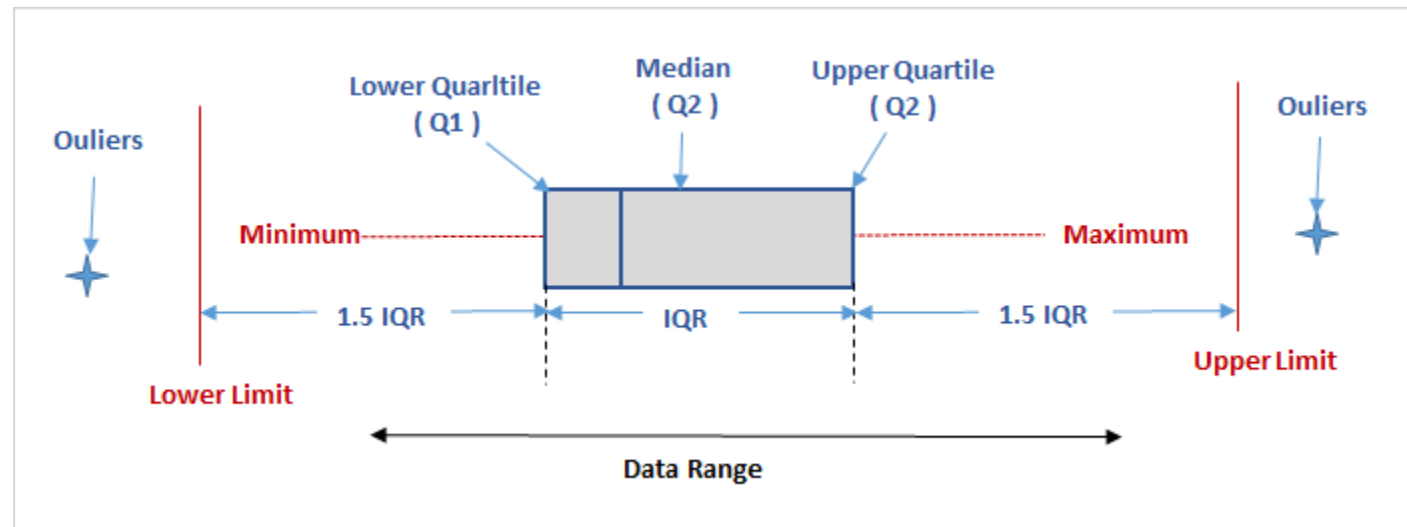
If  $Q1 = 10$  and  $Q3 = 20 \rightarrow IQR = 10$

$\rightarrow$  Lower bound =  $10 - 1.5 \times 10 = -5$

$\rightarrow$  Upper bound =  $20 + 1.5 \times 10 = 35$

$\rightarrow$  Any value outside  $(-5, 35)$  = outlier

# Understanding Outliers and the IQR Method



# Detecting Outliers — Code & Visualization

```
# Identify outliers in the 'fare' column
```

```
Q1, Q3 = df['fare'].quantile([0.25, 0.75])
```

```
IQR = Q3 - Q1
```

```
# Define bounds
```

```
lower_bound = Q1 - 1.5 * IQR
```

```
upper_bound = Q3 + 1.5 * IQR
```

```
# Filter outliers
```

```
outliers = df[(df['fare'] < lower_bound) | (df['fare'] > upper_bound)]
```

```
print("Number of outliers:", len(outliers))
```

# Visualizing Outliers:

```
plt.figure(figsize=(6,4))  
plt.boxplot(df['fare'], vert=False)  
plt.title("Fare Distribution with Outliers")  
plt.xlabel("Fare")  
plt.show()
```

- Boxplots are ideal for spotting outliers visually.
- Whiskers represent limits ( $Q1 - 1.5 \times IQR$  and  $Q3 + 1.5 \times IQR$ ).
- Dots outside whiskers = potential outliers.

“The IQR method helps us detect unusual values — use visualization to decide what to do next.”

# Crosstab Analysis

## Concept:

- **Crosstab** = Cross-tabulation → A **frequency table** showing the relationship between **two or more categorical variables**.
- Used to **summarize counts or proportions** in a dataset.
- Similar to a **pivot table** in Excel.

## Purpose:

- To explore **interactions** between variables like gender & survival, region & education, etc.
- Helps detect **patterns and associations** between categories.

## Basic Syntax:

```
pd.crosstab(df['sex'], df['survived'])
```

## Output

| sex    | 0 (Died) | 1 (Survived) |
|--------|----------|--------------|
| female | 81       | 233          |
| male   | 468      | 109          |

# Normalizing Crosstabs — Proportion Analysis

- **Concept:**

Raw counts can be misleading when groups are of different sizes.

✅ **Normalize** the crosstab to get proportions instead of absolute numbers.

**Code Example:**

```
pd.crosstab(df['sex'], df['survived'], normalize='index')
```

- **Key Point:**

- Each row sums to 1 (or 100%).
- Easier to compare categories fairly.
- “Normalization shows proportions, not counts.”

**Example Output (display as a table):**

| sex    | 0      | 1          |
|--------|--------|------------|
|        | (Died) | (Survived) |
| female | 0.26   | 0.74       |
| male   | 0.81   | 0.19       |

# Crosstab with Multiple Variables & Visualization

## Adding More Dimensions:

You can include a third variable (like passenger class) for deeper insights.

## Code Example:

```
pd.crosstab([df['sex'], df['pclass']], # row categories
            df['survived'],
            normalize='index'
)
```

Sample Output (display condensed):

| sex    | pclass | Died | Survived |
|--------|--------|------|----------|
| female | 1      | 0.03 | 0.97     |
| female | 3      | 0.49 | 0.51     |
| male   | 1      | 0.37 | 0.63     |
| male   | 3      | 0.83 | 0.17     |

# Visualizing Crosstab

- A **heatmap** using Seaborn:

```
sns.heatmap(pd.crosstab(df['sex'], df['survived'], normalize='index'), annot=True, cmap='coolwarm')  
plt.title("Survival Proportion by Gender")  
plt.show()
```

- **Insights:**
- Multi-variable crosstabs help identify combined effects (e.g., gender *and* class influence survival).
- Visuals make proportions more intuitive to grasp.



# Grouping and Aggregation

## Concept:

- The `groupby()` function splits data into **groups** based on a categorical variable.
- The `agg()` (aggregate) method allows applying **multiple summary functions** to one or more columns.
- **Analogy:**  
Think of it as “creating a summary table” — like an Excel pivot table, but more flexible.
- **Basic Syntax:**
- `df.groupby('pclass')['age'].mean()`
- Groups passengers by **pclass** and calculates the **average age** in each class.

| pclass | mean_age |
|--------|----------|
| 1      | 38.2     |
| 2      | 29.9     |
| 3      | 25.1     |

# Aggregating Multiple Metrics

- **Concept:**  
You can summarize **several variables simultaneously** using a dictionary mapping each column to the desired function(s).

- **Code Example:**

```
df.groupby('pclass').agg({  
    'age': 'mean',  
    'fare': 'mean',  
    'survived': 'mean'  
})
```

| pclass | age  | fare  | survived |
|--------|------|-------|----------|
| 1      | 38.2 | 84.15 | 0.63     |
| 2      | 29.9 | 20.66 | 0.47     |
| 3      | 25.1 | 13.68 | 0.24     |

## Interpretation:

- **Age** and **Fare** decrease from 1st → 3rd class.
- **Survival rate** (mean of 0/1) also declines across classes.

# Multiple Functions per Column + Visualization

- **Concept:**  
You can apply **multiple summary functions** to the same variable — e.g., mean, median, std.

- **Code Example:**

```
df.groupby('pclass').agg({  
    'age': ['mean', 'median'],  
    'fare': ['mean', 'std'],  
    'survived': 'mean'  
})
```

| pclass | age_mean | age_median | fare_mean | fare_std | survived_mean |
|--------|----------|------------|-----------|----------|---------------|
| 1      | 38.2     | 39.0       | 84.15     | 78.0     | 0.63          |
| 2      | 29.9     | 29.0       | 20.66     | 13.5     | 0.47          |
| 3      | 25.1     | 24.0       | 13.68     | 11.3     | 0.24          |

.....

Visual Idea:

```
df.groupby('pclass')['survived'].mean().plot(kind='bar', title='Survival Rate by Passenger  
Class')  
plt.ylabel('Survival Rate')  
plt.show()
```

Key Takeaway:

Grouping and aggregation reveal patterns and differences across categories, forming the backbone of exploratory data analysis.

# Analyzing Categorical Distributions

- **What Are Categorical Distributions?**

**Concept:**

- Categorical variables represent **groups or labels** (e.g., gender, region, class).
- Understanding how data is distributed across these categories helps uncover **patterns or imbalances**.
- Useful for identifying:
  - ✓ Which categories dominate your dataset
  - ✓ Whether missing or rare categories exist
  - ✓ How categories relate to outcomes (like survival)

- **Example:**

```
df['embarked'].value_counts()
```

| Embarked | Count |
|----------|-------|
| S        | 644   |
| C        | 168   |
| Q        | 77    |

# Visualizing Categorical Counts

- **Using Seaborn's `countplot()` for Category Analysis**
- **Concept:**  
A **count plot** shows how often each category occurs — optionally broken down by another variable.
- **Code Example:**

```
import seaborn as sns  
sns.countplot(x='embarked', data=df)  
plt.title('Passenger Count by Embarkation Port')  
plt.xlabel('Embarkation Port')  
plt.ylabel('Number of Passengers')  
plt.show()
```

## **Visual Description:**

 Bar chart showing counts for **S**, **C**, and **Q**.

## **Takeaway:**

- The majority embarked from **S**.
- Visual imbalance alerts you to potential **sampling bias**.
- Try adding `hue='sex'` to show gender differences:
- `sns.countplot(x='embarked', hue='sex', data=df)`

# Comparing Categories by Outcome

Comparing Categorical Variables vs. Survival Concept: Add a hue parameter to explore relationships between categories and a target variable.

## Code Example:

```
sns.countplot(x='embarked', hue='survived', data=df)
plt.title('Survival by Embarkation Port')
plt.xlabel('Embarkation Port')
plt.ylabel('Passenger Count')
plt.legend(title='Survived', labels=['No', 'Yes'])
plt.show()
```

## Interpretation:

- Passengers from C had the highest survival proportion.
- Those from S had the lowest — suggesting possible link to class distribution.

## Key Learning:

EDA visualizations help us spot trends quickly before statistical testing.

# Comparing Two Numerical Variables

- **Why Compare Two Numeric Variables?**
- **Concept:**
- Many datasets have numeric features that may **influence each other**.  
Examples:
  - Age vs. Fare
  - Income vs. Education years
  - Height vs. Weight
- Comparing them helps identify **linear trends, clusters, or outliers**.
- **Example (Titanic dataset):**
- `df[['age', 'fare']].head()`

| age | fare  |
|-----|-------|
| 22  | 7.25  |
| 38  | 71.28 |
| 26  | 7.92  |
| 35  | 53.10 |
| 35  | 8.05  |



# Visualizing Relationships — Scatter Plot with Regression

## Seaborn's Implot() — Linear Relationship Visualization

### Concept:

- A scatter plot shows each observation as a point.
- The regression line shows the line of best fit — visualizing overall trend.

### Code Example:

```
import seaborn as sns
sns.lmplot(x='age', y='fare', data=df)
plt.title('Age vs Fare — Regression Line')
plt.xlabel('Age')
plt.ylabel('Fare')
plt.show()
```

### Visual Description:

Dots represent individual passengers.

The blue line represents the estimated linear trend (fit).

Shaded area = confidence interval around the regression line.

### Key Learning:

Even before formal regression, EDA allows you to see the pattern

# Interpreting the Relationship

## What Does the Regression Line Tell Us?

### Concept:

- **Positive slope:** as one variable increases, the other tends to increase.
- **Negative slope:** one increases, the other decreases.
- **Flat line:** no clear relationship.

### Advanced Exploration:

You can segment by category to add context:

```
sns.lmplot(x='age', y='fare', hue='sex', data=df)
plt.title('Age vs Fare by Gender')
plt.show()
```

### Interpretation Example:

- Males and females may show different fare–age relationships.
- Suggests an interaction between **categorical and numerical variables**.

### Takeaway:

EDA reveals **real-world patterns** — guiding what to test statistically later.

- 
- END of CHApter 1

# Covariance vs Correlation

- `df[['age','fare']].cov()`
- `df[['age','fare']].corr()`

| Concept     | Meaning                     | Range      |
|-------------|-----------------------------|------------|
| Covariance  | How variables move together | Any number |
| Correlation | Scaled covariance           | -1 to +1   |

# Probability Refresher

## **Concept:**

- Probability = chance of an event happening
- Between 0 and 1

## **Code:**

```
import numpy as np
events = np.random.choice(['Heads', 'Tails'], 1000)
(np.array(events) == 'Heads').mean()
```

“Use this to show simulation-based probability.”

# Probability Distributions

## **Concept:**

- Visualize how data is spread.
- Example: Normal, Uniform, Binomial

## **Code:**

```
data = np.random.normal(0, 1, 1000)
sns.histplot(data, kde=True)
plt.title("Normal Distribution")
```

# Standard Normal Distribution & z-scores

The **Z-score** standardizes data — expressing how far each value is from the **mean** in terms of **standard deviations**

Formula:

$$z = \frac{(x - \mu)}{\sigma}$$

Where:

$x$  = individual data point

$\mu$  = mean

$\sigma$  = standard deviation

# .. Standard Normal Distribution & z-scores

## Code:

```
z = (df['age'] - df['age'].mean()) / df['age'].std()
```

## Key Idea:

- Z-scores convert any numeric column into a **standard normal scale**.
- Most data lies within:
  - $\pm 1\sigma \rightarrow 68\%$
  - $\pm 2\sigma \rightarrow 95\%$
  - $\pm 3\sigma \rightarrow 99.7\%$
- **Visual Description:**  
A bell curve (normal distribution) showing most data centered near 0.



# Visualizing Standardized Data

- **Code:**

```
import seaborn as sns
sns.histplot(z, kde=True, color='teal')
plt.title('Distribution of Z-Scores for Age')
plt.xlabel('Z-score')
plt.ylabel('Frequency')
plt.show()
```

**Interpretation:**

- The curve should resemble a **bell shape** centered near 0.
- Points far from the center (left or right) are **potential outliers**.
- The **KDE (Kernel Density Estimate)** smooths the histogram for better visualization.
- **Visual Description:**
  - Most z-scores between -2 and +2.
  - A few points beyond  $\pm 3$  indicate **extreme ages** in the dataset.

# Detecting Outliers Using Z-scores

```
outliers = df[abs(z) > 3]
print(outliers[['age']])
```

## Concept:

- Any data point with  $|z| > 3$  is considered **an outlier** (beyond 3 standard deviations).
- This method assumes data follows a **normal distribution**.

Compare this method to IQR-based detection

| Method  | Basis                | Threshold               | Best For                 |
|---------|----------------------|-------------------------|--------------------------|
| IQR     | Spread between Q1–Q3 | $1.5 \times \text{IQR}$ | Skewed data / non-normal |
| Z-score | Standard deviation   |                         | z                        |

.....

### Interpretation Example:

- In Titanic data, very young or very old passengers may appear as z-score outliers.
- But for skewed variables (like fare), **IQR** might be more reliable.

### Takeaway:

Z-score and IQR both detect outliers — but choice depends on **data distribution**.

# Introduction to Central Limit Theorem (CLT)

## What is the Central Limit Theorem?

### Concept:

- The **Central Limit Theorem (CLT)** states that:
- When you take many random samples from any population and calculate their means, the **distribution of those means** tends to become **normal (bell-shaped)** — **even if the original data isn't normally distributed.**

### Key Points:

- Works for **large sample sizes** (typically  $n \geq 30$ ).
- The mean of the sampling distribution  $\approx$  population mean.
- The spread of the sampling distribution decreases as sample size increases.

### Why It Matters:

- CLT is the foundation of **statistical inference** — it allows us to estimate population parameters from samples.
- Enables **confidence intervals, hypothesis testing, and p-value calculations**

# Demonstrating the CLT in Python

- **CLT in Action — Sampling the Titanic Fares**

- **Code Example:**

```
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
# Step 1: Draw random samples and compute their means
means = [np.mean(np.random.choice(df['fare'].dropna(), 50)) for _ in range(1000)]
# Step 2: Plot the sampling distribution
sns.histplot(means, kde=True, color='skyblue')
plt.title("Sampling Distribution of Fare Means (n=50, 1000 samples)")
plt.xlabel("Sample Mean Fare")
plt.ylabel("Frequency")
plt.show()
```

# .. Demonstrating the CLT in Python

## Explanation:

- `np.random.choice()` randomly samples 50 fares **with replacement**.
- `np.mean()` computes the mean of each sample.
- The loop runs 1000 times to create a **distribution of sample means**.
- Finally, we visualize this distribution using Seaborn.

## Visual Description:

Even though fare in Titanic is **right-skewed** (some passengers paid much more),  
the **sampling means** histogram looks **bell-shaped** — that's CLT in action.

# Why the CLT is Important

- Understanding the Power of the CLT

Concept Summary Table:

| Element               | Meaning                                     | Example (Titanic Data)                          |
|-----------------------|---------------------------------------------|-------------------------------------------------|
| Population            | All passenger fares                         | All <code>fare</code> values in <code>df</code> |
| Sample                | Subset of passengers                        | 50 random fares                                 |
| Sampling Distribution | Distribution of sample means                | The <code>means</code> list we created          |
| Result                | Sampling means follow a normal distribution | Shown in histogram                              |

## Key Takeaways:

- CLT allows us to make probabilistic statements about sample means.
- With a large enough sample, we can assume normality, even if data is skewed.
- The standard error (spread of sample means) becomes smaller as sample size grows.

## Visual Suggestion:

Show 3 small histograms:

1. Original fare data (skewed right)
2. Sample means with  $n=10$  (still irregular)
3. Sample means with  $n=100$  (smooth bell curve)

# Correlation vs. Causation – The Core Idea

## Correlation Does Not Imply Causation

### Concept:

- **Correlation** measures the *strength and direction* of a relationship between two variables.
- But even a **strong correlation** does **not** mean one variable **causes** the other.

### Example:

- A study finds that **ice cream sales** and **drowning rates** rise together.

### Why?

Because both increase during **hot weather** — a *confounding variable*.

### Visual (suggested for slide):

- A simple scatter plot with both lines trending upward.
- Add a third icon (🌞 “Hot Weather”) in the middle to illustrate the hidden link



# Understanding Confounding Variables

- **The Hidden Link — Confounding Factors**
- **Concept:**  
A **confounding variable** is a hidden factor that influences both variables, creating a **false or misleading relationship**.

| Variable A        | Variable B           | Confounder      |
|-------------------|----------------------|-----------------|
| Ice cream sales 🍦 | Drowning incidents 🏊 | Temperature ☀️  |
| Coffee intake ☕   | Productivity 💼       | Sleep quality 😴 |
| Shoe size 👟       | Reading ability 📖    | Age 🧑 ➡️ 🧑 ➡️ 🧑 |

# ...Understanding Confounding Variables

- **Code Demo Idea :**  
Simulate spurious correlation:

```
import pandas as pd
```

```
import numpy as np
```

```
np.random.seed(42)
```

```
temperature = np.random.randint(20, 40, 100)
```

```
ice_cream_sales = temperature * 10 + np.random.randint(0, 30, 100)
```

```
drowning_cases = temperature * 2 + np.random.randint(0, 5, 100)
```

```
df = pd.DataFrame({'temperature': temperature,  
                  'ice_cream_sales': ice_cream_sales,  
                  'drowning_cases': drowning_cases})
```

```
print(df.corr())
```

# Why This Matters in Data Analysis

## Interpreting Correlations Responsibly

### Key Takeaways:

- Correlation helps **discover patterns**, but not **prove causes**.
- Always check for **external or hidden factors**.
- Use **domain knowledge**, not just statistics, to interpret relationships.
- Causation usually requires **controlled experiments** or **longitudinal data**.

### Visual Suggestion:

A triangle diagram:

$A \leftrightarrow B$  (correlation)

\   /

C (confounder)

# Introduction to Hypothesis Testing

## Concept:

Hypothesis testing helps us decide whether observed patterns in data are due to **chance** or reflect a **real effect**.

## Scenario:

“Did women really survive more than men by chance on the Titanic?”

- We use hypothesis testing to **quantify** such questions.

## Key Terms:

- **Null Hypothesis ( $H_0$ )**: No relationship or difference exists.
- **Alternative Hypothesis ( $H_1$ )**: A relationship or difference does exist.

# Formulating the Titanic Example

## Example:

- **Question:** Does gender affect survival rate?
- **$H_0$ :** Survival is independent of gender (any difference is random).
- **$H_1$ :** Survival depends on gender (difference is meaningful).

## Intuition:

If we repeatedly sampled passengers, would the survival gap between men and women still appear — or was it just luck?

# Interpretation (Conceptual, No Math Yet)

## Understanding Results:

- If the observed difference is **too large** to be explained by random variation  $\rightarrow$  reject  $H_0$ .
- Otherwise, we **fail to reject  $H_0$**  (no strong evidence of an effect).

## Takeaway:

- Hypothesis testing helps move from *observation* (“more women survived”) to *inference* (“gender likely influenced survival”).
- This sets the foundation for later statistical tests (e.g., Chi-square test).

# Comparing Two Groups — Concep

## **Goal:**

Test if men and women had significantly different survival rates.

## **Concept:**

The **t-test** compares **means** of two independent groups to check if the difference is statistically significant (not due to random chance).

## **Example Question:**

- Did women survive more than men — and was that difference likely real?

# Running the t-test in Python

## Code Example:

```
from scipy import stats
```

```
male = df[df['sex'] == 'male']['survived']
```

```
female = df[df['sex'] == 'female']['survived']
```

```
stats.ttest_ind(male, female, nan_policy='omit')
```

## Explanation:

- `ttest_ind` → compares two independent samples
- `nan_policy='omit'` → ignores missing values
- Output gives:
  - **t-statistic** → how big the difference is
  - **p-value** → how likely the difference is due to chance



# Interpreting the Results

## Interpretation:

- If  $p < 0.05$ , the difference is **statistically significant**  
→ Survival rates for men and women are **likely different** in reality.
- If  $p \geq 0.05$ , the difference could just be **random variation**.

## In Titanic Data:

Women had a much higher survival rate — and a **very small p-value**, confirming the difference is **real, not by chance**.

## Takeaway:

The t-test turns a visual observation into **statistical evidence** — a bridge from *descriptive* to *inferential* analysis.

# What is a p-value? (Concept)

## Definition:

The **p-value** measures how likely your observed difference could have happened **by random chance** — assuming the **null hypothesis ( $H_0$ )** is true.

## Key Idea:

- **High p-value ( $> 0.05$ ):** Difference could be random
- **Low p-value ( $< 0.05$ ):** Unlikely due to chance → likely a real effect

## Analogy:

Think of it as the *probability of surprise* —  
“How surprised should we be if nothing’s really different?”

# Visual Understanding

**Visual:** Two overlapping bell curves

- One curve = Group A (e.g., men's survival)
- Other curve = Group B (e.g., women's survival)
- **Overlap area = p-value region**

**Interpretation:**

If the overlap (p-value) is **small**, the two groups are **different enough** that the difference is statistically meaningful.

```
sns.kdeplot(male, label='Male')
```

```
sns.kdeplot(female, label='Female')
```

```
plt.legend()
```

# Decision Rule & Interpretation

Decision Rule:

| p-value       | Decision             | Meaning                   |
|---------------|----------------------|---------------------------|
| $p < 0.05$    | Reject $H_0$         | Significant difference    |
| $p \geq 0.05$ | Fail to reject $H_0$ | No significant difference |

## Example (Titanic):

If  $p = 0.001 \rightarrow$  Women's higher survival rate is **very unlikely by chance**.

✓ **We reject  $H_0$**   $\rightarrow$  Gender mattered in survival outcomes.

# Visualizing the Meaning of a p-value

## Concept Recap

- Each curve shows the **distribution of survival rates** (simulated for men vs women).
  - The **shaded overlap** area represents where the two groups' values are **not significantly different** — the p-value region.
  - When this overlap is small ( $p < 0.05$ ), we conclude that the difference is **statistically significant**.
- 
- “The more these curves overlap, the higher the p-value — meaning less evidence of a real difference.”
  - “If the curves are far apart (small overlap), the p-value becomes small — evidence the groups differ.”
  - “We don’t ‘prove’ anything with a p-value; we just assess whether the observed difference is likely due to chance.”

# Python Demo Code (Interactive Visual)

```
import numpy as np

import seaborn as sns

import matplotlib.pyplot as plt

from scipy import stats

# Generate two simulated groups (e.g., survival scores)

np.random.seed(42)

male = np.random.normal(0.45, 0.10, 200) # mean 45%, some spread

female = np.random.normal(0.70, 0.10, 200) # mean 70%, some spread

# Conduct t-test

t_stat, p_value = stats.ttest_ind(male, female)

print(f"t-statistic = {t_stat:.2f}, p-value = {p_value:.4f}")

# Plot overlapping distributions

sns.kdeplot(male, label='Male', fill=True, alpha=0.4)

sns.kdeplot(female, label='Female', fill=True, alpha=0.4)

# Annotate chart

plt.title(f"Visualizing p-value (p = {p_value:.4f})")

plt.xlabel("Survival Rate")

plt.ylabel("Density")

plt.legend()

# Highlight p-value interpretation zone

plt.text(0.5, 2.0, "Overlap = possible by chance (p-value region)", fontsize=10, color='gray')

plt.text(0.72, 1.5, "Separated = likely real difference", fontsize=10, color='darkgreen')

plt.show()
```

....

## Slide Summary

- **p-value < 0.05:** Overlap small → difference is statistically significant
- **p-value ≥ 0.05:** Overlap large → difference could be random

# Recap

- Clean data before analysis
- Understand variable relationships
- Probability connects data  $\rightarrow$  inference
- Hypothesis testing formalizes findings